

PROBLEM 1: Check Age Category. Write a java program with a special class

- To store Name and DOB
- To calculate Age
- To specify Age Category as follows:
- "Under Age", if Age < 18
- "Young Adult", if 18 < Age < 30
- "Matured Adult", if 30 < Age < 45
- Middle Aged, if 45 < Age < 60 & so on...

Source Code:

```
import java.util.Scanner;
import java.util.Calendar;

class Person_1 {
    String name;
    int day, month, year;
    Calendar today = Calendar.getInstance();
    int currentYear = today.get(Calendar.YEAR);
    int currentMonth = today.get(Calendar.MONTH) + 1;
    int currentDay = today.get(Calendar.DAY_OF_MONTH);
    Person_1(String name, int d, int m, int y) {
        this.name = name;
        this.day = d;
        this.month = m;
        this.year = y;
    }
    public boolean isValidDate() {
        if (month < 1 || month > 12) return false;
        int[] daysInMonth = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
            daysInMonth[2] = 29;
        }
        if (day <= 0 || day > daysInMonth[month]) return false;
        Calendar dob = Calendar.getInstance();
        dob.set(year, month - 1, day);
        if (dob.after(today)) return false;
        return true;
    }
    public int calculateAge() {
        int age = currentYear - year;
        if (currentMonth < month || (currentMonth == month && currentDay < day)) {
            age--;
        }
        return age;
    }
    public String getAgeCategory() {
        int age = calculateAge();
        if (age < 18) return "Under Age.";
        if (age < 30) return "Young Adult.";
        if (age < 45) return "Matured Adult.";
        if (age < 60) return "Middle Aged.";
        return "Senior.";
    }
    public void display() {
        if (!isValidDate()) {
            System.out.println("Error: The date " + day + "/" + month + "/" + year + " is invalid or in the future.");
        }
    }
}
```

```

        return;
    }
    System.out.println("\nName: " + name);
    System.out.println("\nAge: " + calculateAge());
    System.out.println("\nCategory: " + getAgeCategory());
}
}

public class Q1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.println("Enter name: ");
            String name = sc.nextLine();
            System.out.println("Enter DOB (Day Month Year): ");
            int d = sc.nextInt();
            int m = sc.nextInt();
            int y = sc.nextInt();
            Person_1 p = new Person_1(name, d, m, y);
            p.display();
        } catch (Exception e) {
            System.out.println("Error: Enter valid numeric values for the date.");
        } finally {
            sc.close();
        }
    }
}

```

Sample Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass1$ java Q1.java
Enter name:
Bappa Das
Enter DOB (Day Month Year):
23 05 2004

Name: Bappa Das

Age: 22

Category: Young Adult.

```

```

bappa@fedora:~/MCA_1year_2sem/Java/ass1$ java Q1.java
Enter name:
Anubav
Enter DOB (Day Month Year):
23 07 2026
Error: The date 23/7/2026 is invalid or in the future.

```

PROBLEM 2: Design your own STACK. Such that the Stack class objects need to be:

- Initialised by a Constructor
- Provided proper push() and pop() methods
- Checked for TOS content using peek()
- Overloaded with display() for several tasks
 - Display content of whole stack
 - Given depth of a particular element, show the element

Source Code:

Stack.java

```

package mystack;
public class Stack {
    protected int [] arr;
    protected int top;
    protected int size;
    public Stack(int size){

```

```

        this.size = size;
        arr = new int[size];
        top = -1;
    }
    public boolean push(int value){
        if(top==size-1){
            return false;
        }else{
            top++;
            arr[top] = value;
            return true;
        }
    }
    public int size(){
        return top+1;
    }
    public int pop(){
        if(top== -1){
            return -1;
        }else{
            int val = arr[top];
            top--;
            return val;
        }
    }
    public int peek(){
        if(top== -1){
            return -1;
        }else{
            return arr[top];
        }
    }
    public void display(){
        if(top== -1){
            System.out.println("Stack is empty!");
        }else{
            System.out.println("Stack elements: ");
            for (int i=top; i>=0;i--){
                System.out.println(arr[i]);
            }
        }
    }
    public boolean isEmpty() {
        return top == -1;
    }
    public boolean isFull() {
        return top == size - 1;
    }
    public int display(int depth){
        if (depth < 0 || depth > top) {
            return -1;
        } else {
            return arr[top - depth];
        }
    }
}

```

Q2.java

```
import java.util.Scanner;
import mystack.Stack;
public class Q2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Stack Size: ");
        int n = sc.nextInt();
        Stack s = new Stack(n);
        int choice;
        do {
            System.out.println("\n--- STACK MENU ---");
            System.out.println("1. Push");
            System.out.println("2. Pop");
            System.out.println("3. Peek");
            System.out.println("4. Display Full Stack");
            System.out.println("5. Display Element by Depth");
            System.out.println("6. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Value: ");
                    int value = sc.nextInt();
                    if (s.push(value)) {
                        System.out.println(value + " pushed in the stack.");
                    } else {
                        System.out.println("Stack overflow!");
                    }
                    break;
                case 2:
                    int popped = s.pop();
                    if (popped == -1) {
                        System.out.println("Stack underflow!");
                    } else {
                        System.out.println(popped + " popped from the stack.");
                    }
                    break;
                case 3:
                    int topElement = s.peek();
                    if (topElement == -1) {
                        System.out.println("Stack is empty!");
                    } else {
                        System.out.println("Top element: " + topElement);
                    }
                    break;
                case 4:
                    s.display();
                    break;
                case 5:
                    System.out.print("Enter Depth: ");
                    int depth = sc.nextInt();
                    if (s.isEmpty()) {
                        System.out.println("Stack is empty!");
                    } else {
                        int element = s.display(depth);
                        if (element == -1) {
```

```

        System.out.println("Invalid Depth!");
    } else {
        System.out.println("Element at depth " + depth + " is: " + element);
    }
}
break;
case 6:
    System.out.println("Exiting Program...");
    break;
default:
    System.out.println("Invalid Choice!");
}

} while (choice != 6);
sc.close();
}
}

```

Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac mystack/Stack.java Q2.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ java Q2.java
Enter Stack Size: 3

--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 2
Stack underflow!

--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 1
Enter Value: 1
1 pushed in the stack.

--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 1
Enter Value: 2
2 pushed in the stack.

--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 1
Enter Value: 3
3 pushed in the stack.

```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 1
Enter Value: 4
Stack overflow!
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 3
Top element: 3
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 4
Stack elements:
3
2
1
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 5
Enter Depth: 5
Invalid Depth!
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 5
Enter Depth: 2
Element at depth 2 is: 1
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display Full Stack
5. Display Element by Depth
6. Exit
Enter Choice: 6
Exiting Program...
```

PROBLEM 3: Implement Merge operation on two Stacks presorted in ascending order. Check for stack overflow conditions all along.

PROBLEM 4: Inherit a special stack. The inherited class should allow

- A number to be pushed in if and only if it is less than the one currently at the top
- With the help of such stack objects, solve the Tower of Hanoi problem most economically, by arranging a sequence of haphazard numbers in the original base type Stack in ascending order in a final STACK.

Source Code:

SpecialStack.java

```
package mystack;
public class SpecialStack extends Stack{

    public SpecialStack(int size){
        super(size);
    }

    public boolean push(int val){
        if (isFull()){
            System.out.println("Stack Overflow!");
            return false;
        }
        if (isEmpty() || val <= peek()){
            arr[++top] = val;
            return true;
        }else{
            return false;
        }
    }
}
```

StackSorter.java

```
package mystack;
public class StackSorter {
    public static SpecialStack sortStack(Stack A) {
        // Stack A = new Stack();
        Stack B = new Stack(A.size());
        SpecialStack C = new SpecialStack(A.size());

        System.out.println("Initial Stack :");
        A.display();
        System.out.println("-----");

        while (!A.isEmpty()){
            int x = A.peek();
            while (!C.isEmpty() && x > C.peek()){
                B.push(C.pop());
            }
            C.push(A.pop());
            while (!B.isEmpty()){
                C.push(B.pop());
            }
        }
        return C;
    }
}
```

Q3.java

```
import java.util.Scanner;
import mystack.SpecialStack;
import mystack.Stack;
import mystack.StackSorter;

public class Q3 {
    static SpecialStack merge(SpecialStack s1, SpecialStack s2, int totalSize) {
        Stack temp = new Stack(totalSize);
        while (!s1.isEmpty() && !s2.isEmpty()) {
            if (s1.peek() < s2.peek()) {
                temp.push(s1.pop());
            } else {
                temp.push(s2.pop());
            }
        }
        while (!s1.isEmpty()) {
            temp.push(s1.pop());
        }
        while (!s2.isEmpty()) {
            temp.push(s2.pop());
        }
        SpecialStack result = new SpecialStack(totalSize);
        while (!temp.isEmpty()) {
            result.push(temp.pop());
        }
        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the size of first Array:");
        int a = sc.nextInt();
        Stack s1 = new Stack(a);
        System.out.println("Enter the elements to push in one by one:");
        while (s1.size() < a) {
            s1.push(sc.nextInt());
        }
        SpecialStack ss1 = StackSorter.sortStack(s1);
        System.out.println("Enter the size of second Array:");
        int b = sc.nextInt();
        Stack s2 = new Stack(b);
        System.out.println("Enter the elements to push in one by one:");
        while (s2.size() < b) {
            s2.push(sc.nextInt());
        }
        SpecialStack ss2 = StackSorter.sortStack(s2);
        System.out.println("Stack 1:");
        ss1.display();
        System.out.println("Stack 2:");
        ss2.display();
        SpecialStack merged = merge(ss1, ss2, 20);
        System.out.println("\nMerged Special Stack:");
        merged.display();
        sc.close();
    }
}
```


Output:

```
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac mystack/SpecialStack.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac mystack/StackSorter.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac Q3.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ java Q3.java
Enter the size of first Array:
3
Enter the elements to push in one by one:
6
2
5
Initial Stack :
Stack elements:
5
2
6
-----
Enter the size of second Array:
2
Enter the elements to push in one by one:
9
7
Initial Stack :
Stack elements:
7
9
-----
Stack 1:
Stack elements:
2
5
6
Stack 2:
Stack elements:
7
9
-----
Merged Special Stack:
Stack elements:
2
5
6
7
9
```

Q4.java

```
import java.util.Scanner;
import mystack.SpecialStack;
import mystack.Stack;
import mystack.StackSorter;

public class Q4 {
    public static void main(String[] args) {
        int n = 0;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of elements:");
        n = sc.nextInt();
        Stack A = new Stack(n);
        System.out.print("Enter elements in Stack: ");
        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
            A.push(num);
        }
    }
}
```

```

    }
    SpecialStack res = StackSorter.sortStack(A);
    System.out.println("-----");
    System.out.println("Final Sorted SpecialStack C: ");
    res.display();
    sc.close();
}
}

```

Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac mystack/SpecialStack.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac mystack/StackSorter.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ javac Q4.java
bappa@fedora:~/MCA_1year_2sem/Java/ass1$ java Q4.java
Enter the number of elements:
4
Enter elements in Stack: 6 7 2 9
Initial Stack :
Stack elements:
9
2
7
6
-----
-----
Final Sorted SpecialStack C:
Stack elements:
2
6
7
9

```

PROBLEM 5: Create a new class Graded_Emp from an existing class Emp, where class Emp has got two fields ID and Dep_Code[0-5], whereas the derived class has an additional field Grade[A-E].

PROBLEM 6: Use the super Keyword judiciously to (a) populate objects of both the base and derived classes above with minimum repetition of code & (b) change Dep_Code 0 to 10 while populating derived objects.

PROBLEM 7: Create your own exception class to trap accepting invalid data above.

PROBLEM 8: How would you restructure your code if the Grade field is dependent on average marks obtained in several skill-tests recorded separately?

Source Code:

```

import java.util.Arrays;
import java.util.Scanner;
class InvalidEmployeeDataException extends Exception {
    public InvalidEmployeeDataException(String message) {
        super(message);
    }
}

class Emp {
    protected int id;

```

```

protected int depCode;
public Emp(int id, int depCode) throws InvalidEmployeeDataException {
    if (depCode < 0 || depCode > 5) {
        throw new InvalidEmployeeDataException("Invalid Base Employee Department Code: " + depCode + ". Allowed
range is 0 to 5.");
    }
    this.id = id;
    this.depCode = depCode;
}
protected Emp(int id, int depCode, int maxRange) throws InvalidEmployeeDataException {
    if (depCode < 0 || depCode > maxRange) {
        throw new InvalidEmployeeDataException("Invalid Department Code: " + depCode + ". Allowed range is 0 to " +
maxRange + ".");
    }
    this.id = id;
    this.depCode = depCode;
}
public void display() {
    System.out.print("Emp ID: " + id + " | Dep Code: " + depCode);
}
}

class GradedEmp extends Emp {
    private double[] skillTestMarks;
    public GradedEmp(int id, int depCode, double[] skillTestMarks) throws InvalidEmployeeDataException {
        super(id, depCode == 0 ? 10 : depCode, 10);
        if (skillTestMarks == null || skillTestMarks.length == 0) {
            throw new InvalidEmployeeDataException("Skill test marks cannot be null or empty.");
        }
        for (double mark : skillTestMarks) {
            if (mark < 0 || mark > 100) {
                throw new InvalidEmployeeDataException("Invalid test mark discovered: " + mark + ". Must be between 0 and
100.");
            }
        }
        this.skillTestMarks = skillTestMarks;
    }
    public char calculateGrade() {
        double sum = 0;
        for (double mark : skillTestMarks) {
            sum += mark;
        }
        double average = sum / skillTestMarks.length;
        if (average >= 85) return 'A';
        else if (average >= 70) return 'B';
        else if (average >= 55) return 'C';
        else if (average >= 40) return 'D';
        else return 'E';
    }
    @Override
    public void display() {
        super.display();
    }
}

```

```

        System.out.println(" | Marks: " + Arrays.toString(skillTestMarks) + " | Grade: " + calculateGrade());
    }
}

public class Q5 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean running = true;
        System.out.println("=====");
        System.out.println(" Employee & Graded Employee Verification System ");
        System.out.println("=====");
        while (running) {
            System.out.println("\nSelect a Test Scenario:");
            System.out.println("1) Test Standard Employee Class (Emp)");
            System.out.println("2) Test Graded Employee Class (GradedEmp - with dynamic grading)");
            System.out.println("3) Exit");
            System.out.print("Enter choice (1-3): ");
            int choice = scanner.nextInt();
            switch (choice) {
                case 1:
                    System.out.println("\n--- [Scenario 1: Standard Employee] ---");
                    System.out.print("Enter Employee ID: ");
                    int empId = scanner.nextInt();
                    System.out.print("Enter Department Code [0-5]: ");
                    int depCode = scanner.nextInt();
                    try {
                        Emp emp = new Emp(empId, depCode);
                        System.out.print(">> Success! Created Object: ");
                        emp.display();
                        System.out.println();
                    } catch (InvalidEmployeeDataException e) {
                        System.out.println(">> Trap Success! Custom Exception Caught: " + e.getMessage());
                    }
                    break;
                case 2:
                    System.out.println("\n--- [Scenario 2: Graded Employee] ---");
                    System.out.print("Enter Employee ID: ");
                    int gEmpId = scanner.nextInt();
                    System.out.print("Enter Department Code [0-5]: ");
                    int gDepCode = scanner.nextInt();
                    System.out.print("How many skill tests did this employee take? ");
                    int totalTests = scanner.nextInt();
                    if (totalTests <= 0) {
                        System.out.println(">> Test count must be greater than 0.");
                        break;
                    }
                    double[] marks = new double[totalTests];
                    for (int i = 0; i < totalTests; i++) {
                        System.out.print(" Enter score for test #" + (i + 1) + " (0-100): ");
                        marks[i] = scanner.nextDouble();
                    }
                    try {

```

```

        GradedEmp gEmp = new GradedEmp(gEmpId, gDepCode, marks);
        System.out.print(">> Success! Created Object: ");
        gEmp.display();
    } catch (InvalidEmployeeDataException e) {
        System.out.println(">> Trap Success! Custom Exception Caught: " + e.getMessage());
    }
    break;
case 3:
    running = false;
    System.out.println("\nExiting System");
    break;
default:
    System.out.println("Invalid selection. Please choose an option between 1 and 3.");
}
System.out.println("\n-----");
}
scanner.close();
}
}

```

Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac Q5.java
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ java Q5.java
=====
Employee & Graded Employee Verification System
=====

Select a Test Scenario:
1) Test Standard Employee Class (Emp)
2) Test Graded Employee Class (GradedEmp - with dynamic grading)
3) Exit
Enter choice (1-3): 1

--- [Scenario 1: Standard Employee] ---
Enter Employee ID: 125
Enter Department Code [0-5]: 1
>> Success! Created Object: Emp ID: 125 | Dep Code: 1

-----

Select a Test Scenario:
1) Test Standard Employee Class (Emp)
2) Test Graded Employee Class (GradedEmp - with dynamic grading)
3) Exit
Enter choice (1-3): 1

--- [Scenario 1: Standard Employee] ---
Enter Employee ID: 165
Enter Department Code [0-5]: 0
>> Success! Created Object: Emp ID: 165 | Dep Code: 0

-----

Select a Test Scenario:
1) Test Standard Employee Class (Emp)
2) Test Graded Employee Class (GradedEmp - with dynamic grading)
3) Exit
Enter choice (1-3): 2

--- [Scenario 2: Graded Employee] ---
Enter Employee ID: 67
Enter Department Code [0-5]: 2
How many skill tests did this employee take? 3
Enter score for test #1 (0-100): 67
Enter score for test #2 (0-100): 68
Enter score for test #3 (0-100): 69
>> Success! Created Object: Emp ID: 67 | Dep Code: 2 | Marks: [67.0, 68.0, 69.0] | Grade: C

-----

```

```

Select a Test Scenario:
1) Test Standard Employee Class (Emp)
2) Test Graded Employee Class (GradedEmp - with dynamic grading)
3) Exit
Enter choice (1-3): 2

--- [Scenario 2: Graded Employee] ---
Enter Employee ID: 654
Enter Department Code [0-5]: 0
How many skill tests did this employee take? 2
  Enter score for test #1 (0-100): 69
  Enter score for test #2 (0-100): 98
>> Success! Created Object: Emp ID: 654 | Dep Code: 10 | Marks: [69.0, 98.0] | Grade: B

-----

Select a Test Scenario:
1) Test Standard Employee Class (Emp)
2) Test Graded Employee Class (GradedEmp - with dynamic grading)
3) Exit
Enter choice (1-3): 3

Exiting System

-----

```

PROBLEM 9: Create an interface Shape with the getArea() method. Create three classes Rectangle, Circle and Triangle that implement the Shape interface the Shape interface. Implement the getArea() method for each of the classes.

Source Code:

Shape.java

```

package shapes;
public interface Shape {
    double getArea();
}

```

Rectangle.java

```

package shapes;

public class Rectangle implements Shape {
    private double length;
    private double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    public double getArea() {
        return length * width;
    }
}

```

Circle.java

```

package shapes;

public class Circle implements Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    public double getArea() {

```

```
        return Math.PI * radius * radius;
    }
}
```

Triangle.java

```
package shapes;
```

```
public class Triangle implements Shape {
    private double base;
    private double height;
    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    public double getArea() {
        return 0.5 * base * height;
    }
}
```

PROBLEM 10: Create a `Animal` interface with a method called `sound()` that takes no arguments and returns `void`. Create a few specific animal classes that implement `Animal` and overrides `sound()` differently for each species.

Source Code:

Animal.java

```
package animals;
```

```
public interface Animal {
    void sound();
}
```

Dog.java

```
package animals;
```

```
public class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks: Woof Woof");
    }
}
```

Cat.java

```
package animals;
```

```
public class Cat implements Animal {
    public void sound() {
        System.out.println("Cat meows: Meow Meow");
    }
}
```

Cow.java

```
package animals;
```

```
public class Cow implements Animal {
    public void sound() {
        System.out.println("Cow moos: Moo Moo");
    }
}
```

PROBLEM 11: Create an interface Sortable with a method sort() that sorts an array of integers in ascending order. Create a class BubbleSort that implements the interface and provides its own version of the sort() method.

Source Code:

Sortable.java

```
package sorting;
```

```
public interface Sortable {  
    void sort(int arr[]);  
}
```

BubbleSort.java

```
package sorting;
```

```
public class BubbleSort implements Sortable {  
    public void sort(int arr[]) {  
        int n = arr.length;  
        for(int i = 0; i < n - 1; i++) {  
            for(int j = 0; j < n - i - 1; j++) {  
                if(arr[j] > arr[j + 1]) {  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
                }  
            }  
        }  
        System.out.println("Sorted Array:");  
        for(int x : arr) {  
            System.out.print(x + " ");  
        }  
        System.out.println();  
    }  
}
```

PROBLEM 12: Create an interface ResizableTray with methods W(int width) and L(int length) that help trays to be resized. Create a class Tray that implements the interface to carry a number of fixed sized plates.

Source Code:

ResizableTray.java

```
package resize;
```

```
public interface ResizableTray {  
    void W(int width);  
    void L(int length);  
}
```

Tray.java

```
package resize;
```

```
public class Tray implements ResizableTray {  
    int width;  
    int length;  
    int plateWidth;  
    int plateLength;  
    public void W(int width) {  
        this.width = width;  
    }  
    public void L(int length) {
```



```

        this.length = length;
    }
    public void setPlateSize(int plateWidth, int plateLength) {
        this.plateWidth = plateWidth;
        this.plateLength = plateLength;
    }
    public int calculatePlateCount() {
        if (width <= 0 || length <= 0 || plateWidth <= 0 || plateLength <= 0) {
            return 0;
        }
        int countNormal = (width / plateWidth) * (length / plateLength);
        int countRotated = (width / plateLength) * (length / plateWidth);
        return Math.max(countNormal, countRotated);
    }
    public void display() {
        System.out.println("Tray Details");
        System.out.println("Tray Width: " + width);
        System.out.println("Tray Length: " + length);
        System.out.println("Plate Width: " + plateWidth);
        System.out.println("Plate Length: " + plateLength);
        System.out.println("Number of Plates that can fit: " + calculatePlateCount());
    }
}

```

PROBLEM 13: nherit an interface ResizableBox from ResizableTray with a new method D(int depth). Create a class NewBox that implements ResizableBox to house a given number of balls of fixed using the overridden resizing methods.

Source Code:

ResizableBox.java

package resize;

```

public interface ResizableBox extends ResizableTray {
    void D(int depth);
}

```

NewBox.java

package resize;

```

public class NewBox implements ResizableBox {
    int width;
    int length;
    int depth;
    int ballWidth;
    int ballLength;
    int ballDepth;
    public NewBox(int width, int length, int depth) {
        this.width = width;
        this.length = length;
        this.depth = depth;
    }
    public void W(int width) {
        this.width = width;
    }
    public void L(int length) {
        this.length = length;
    }
}

```

```

public void D(int depth) {
    this.depth = depth;
}
public void setBallSize(int ballWidth, int ballLength, int ballDepth) {
    this.ballWidth = ballWidth;
    this.ballLength = ballLength;
    this.ballDepth = ballDepth;
}
public int calculateBallCount() {
    if (width <= 0 || length <= 0 || depth <= 0 || ballWidth <= 0 || ballLength <= 0 || ballDepth <= 0) {
        return 0;
    }
    int[][] perms = {
        { ballWidth, ballLength, ballDepth },
        { ballWidth, ballDepth, ballLength },
        { ballLength, ballWidth, ballDepth },
        { ballLength, ballDepth, ballWidth },
        { ballDepth, ballWidth, ballLength },
        { ballDepth, ballLength, ballWidth }
    };
    int maxCount = 0;
    for (int[] perm : perms) {
        int fitWidth = width / perm[0];
        int fitLength = length / perm[1];
        int fitDepth = depth / perm[2];
        maxCount = Math.max(maxCount, fitWidth * fitLength * fitDepth);
    }
    return maxCount;
}
public void display() {
    System.out.println("Box Details");
    System.out.println("Box Width: " + width);
    System.out.println("Box Length: " + length);
    System.out.println("Box Depth: " + depth);
    System.out.println("Ball Width: " + ballWidth);
    System.out.println("Ball Length: " + ballLength);
    System.out.println("Ball Depth: " + ballDepth);
    System.out.println("Number of Balls that can fit: " + calculateBallCount());
}
}

```

Main Java File

Q9.java

```

import java.util.Scanner;
import shapes.*;
import animals.*;
import sorting.*;
import resize.*;

public class Q9 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1. Shape Area");
            System.out.println("2. Animal Sounds");

```

```

System.out.println("3. Bubble Sort");
System.out.println("4. Tray Resize");
System.out.println("5. Box Resize");
System.out.println("0. Exit");
System.out.print("Enter Choice: ");
choice = sc.nextInt();
switch(choice) {
    case 1:
        int shapeChoice;
        System.out.println("\n1. Rectangle");
        System.out.println("2. Circle");
        System.out.println("3. Triangle");
        System.out.print("Choose Shape: ");
        shapeChoice = sc.nextInt();
        switch(shapeChoice) {
            case 1:
                System.out.print("Enter Length: ");
                double l = sc.nextDouble();
                System.out.print("Enter Width: ");
                double w = sc.nextDouble();
                Rectangle r = new Rectangle(l, w);
                System.out.println("Area = " + r.getArea());
                break;
            case 2:
                System.out.print("Enter Radius: ");
                double radius = sc.nextDouble();
                Circle c = new Circle(radius);
                System.out.println("Area = " + c.getArea());
                break;
            case 3:
                System.out.print("Enter Base: ");
                double b = sc.nextDouble();
                System.out.print("Enter Height: ");
                double h = sc.nextDouble();
                Triangle t = new Triangle(b, h);
                System.out.println("Area = " + t.getArea());
                break;
            default:
                System.out.println("Invalid Shape Choice");
        }
        break;
    case 2:
        Animal dog = new Dog();
        Animal cat = new Cat();
        Animal cow = new Cow();
        dog.sound();
        cat.sound();
        cow.sound();
        break;
    case 3:
        System.out.print("Enter Size of Array: ");
        int n = sc.nextInt();
        int arr[] = new int[n];
        System.out.println("Enter Elements:");
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
}

```

```

        BubbleSort bs = new BubbleSort();
        bs.sort(arr);
        break;
    case 4:
        System.out.print("Enter Tray Width: ");
        int trayWidth = sc.nextInt();
        System.out.print("Enter Tray Length: ");
        int trayLength = sc.nextInt();
        System.out.print("Enter Plate Width: ");
        int plateWidth = sc.nextInt();
        System.out.print("Enter Plate Length: ");
        int plateLength = sc.nextInt();
        Tray tray = new Tray();
        tray.L(trayLength);
        tray.W(trayWidth);
        tray.setPlateSize(plateWidth, plateLength);
        tray.display();
        break;
    case 5:
        System.out.print("Enter Box Width: ");
        int boxWidth = sc.nextInt();
        System.out.print("Enter Box Length: ");
        int boxLength = sc.nextInt();
        System.out.print("Enter Box Depth: ");
        int boxDepth = sc.nextInt();
        System.out.print("Enter Ball Width: ");
        int ballWidth = sc.nextInt();
        System.out.print("Enter Ball Length: ");
        int ballLength = sc.nextInt();
        System.out.print("Enter Ball Depth: ");
        int ballDepth = sc.nextInt();
        NewBox box = new NewBox(boxWidth, boxLength, boxDepth);
        box.setBallSize(ballWidth, ballLength, ballDepth);
        box.display();
        break;
    case 0:
        System.out.println("Program Exited");
        break;
    default:
        System.out.println("Invalid Choice");
}

} while(choice != 0);
sc.close();
}
}

```

Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac shapes/Shape.java shapes/Rectangle.java sh
apes/Circle.java shapes/Triangle.java
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac animals/Animal.java animals/Cat.java anima
ls/Dog.java animals/Cow.java
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac sorting/Sortable.java sorting/BubbleSort.j
ava
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac resize/ResizableTray.java resize/Tray.java
resize/ResizableBox.java resize/NewBox.java
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ javac Q9.java
bappa@fedora:~/MCA_1year_2sem/Java/ass2$ java Q9.java

```

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 1

1. Rectangle
 2. Circle
 3. Triangle
- Choose Shape: 1

Enter Length: 6

Enter Width: 7

Area = 42.0

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 1

1. Rectangle
2. Circle
3. Triangle

Choose Shape: 2

Enter Radius: 4

Area = 50.26548245743669

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 1

1. Rectangle
2. Circle
3. Triangle

Choose Shape: 3

Enter Base: 4

Enter Height: 6

Area = 12.0

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 2

Dog barks: Woof Woof

Cat meows: Meow Meow

Cow moos: Moo Moo

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 3

Enter Size of Array: 5

Enter Elements:

9 2 6 4 0

Sorted Array:

0 2 4 6 9

===== MENU =====

1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit

Enter Choice: 4

Enter Tray Width: 67

Enter Tray Length: 62

Enter Plate Width: 6

Enter Plate Length: 7

Tray Details

Tray Width: 67

Tray Length: 62

Plate Width: 6

Plate Length: 7

Number of Plates that can fit: 90

```

===== MENU =====
1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit
Enter Choice: 5
Enter Box Width: 36
Enter Box Length: 67
Enter Box Depth: 25
Enter Ball Width: 3
Enter Ball Length: 6
Enter Ball Depth: 7
Box Details
Box Width: 36
Box Length: 67
Box Depth: 25
Ball Width: 3
Ball Length: 6
Ball Depth: 7
Number of Balls that can fit: 440

===== MENU =====
1. Shape Area
2. Animal Sounds
3. Bubble Sort
4. Tray Resize
5. Box Resize
0. Exit
Enter Choice: 0
Program Exited

```

PROBLEM 14: Write the first 10 Fibonacci numbers to a text file “Fibo.txt”.

PROBLEM 15: Read “Fibo.txt” and append next numbers (<250).

PROBLEM 16: Generate a binary file “Fibo.bin” from “Fibo.txt”.

Source Code:

```

import java.io.*;
import java.util.*;

public class Q14 {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("Fibo.txt");
            int a = 0, b = 1;
            for (int i = 0; i < 10; i++) {
                fw.write(a + " ");
                int c = a + b;
                a = b;
                b = c;
            }
            fw.close();
            System.out.println("First 10 Fibonacci numbers written to Fibo.txt");
            Scanner sc = new Scanner(new File("Fibo.txt"));
            ArrayList<Integer> list = new ArrayList<>();
            while (sc.hasNextInt()) {
                list.add(sc.nextInt());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

    }
    sc.close();
    int n = list.size();
    a = list.get(n - 2);
    b = list.get(n - 1);
    FileWriter appendFile = new FileWriter("Fibo.txt", true);
    while (true) {
        int next = a + b;
        if (next >= 250)
            break;
        appendFile.write(next + " ");
        a = b;
        b = next;
    }
    appendFile.close();
    System.out.println("Numbers less than 250 appended.");
    Scanner readTxt = new Scanner(new File("Fibo.txt"));
    DataOutputStream dos = new DataOutputStream(new FileOutputStream("Fibo.bin"));
    while (readTxt.hasNextInt()) {
        dos.writeInt(readTxt.nextInt());
    }
    readTxt.close();
    dos.close();
    System.out.println("Binary file Fibo.bin created.");
} catch (Exception e) {
    System.out.println(e);
}
}
}

```

Output:

```

bappa@fedora:~/MCA_1year_2sem/Java/ass3$ javac Q14.java
bappa@fedora:~/MCA_1year_2sem/Java/ass3$ java Q14.java
First 10 Fibonacci numbers written to Fibo.txt
Numbers less than 250 appended.
Binary file Fibo.bin created.
bappa@fedora:~/MCA_1year_2sem/Java/ass3$ cat Fibo.txt
0 1 1 2 3 5 8 13 21 34 55 89 144 233 bappa@fedora:~/MCA_1year_2sem/Java/ass3$

```

PROBLEM 17: Create a class Profile to store Name, Age, Average Mark. Allow creation of objects and store information for #n persons, with provision for storing content in “Profile.bin”.

PROBLEM 18: Read “Profile.bin” and display the record for the highest scorer.

Source Code:

```

import java.io.*;
import java.util.*;

class Profile implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private int age;
    private double avgMarks;
    Profile(String name, int age, double avgMarks) {
        this.name = name;
        this.age = age;
        this.avgMarks = avgMarks;
    }
}

```

```

public String getName(){
    return name;
}
public int getAge(){
    return age;
}
public double getAvgMarks(){
    return avgMarks;
}
}

public class Q17 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter number of persons: ");
            int n = sc.nextInt();
            sc.nextLine();
            ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("Profile.bin"));
            for (int i = 1; i <= n; i++) {
                System.out.println("\nPerson " + i);
                System.out.print("Name: ");
                String name = sc.nextLine();
                System.out.print("Age: ");
                int age = sc.nextInt();
                System.out.print("Average Marks: ");
                double marks = sc.nextDouble();
                sc.nextLine();
                Profile p = new Profile(name, age, marks);
                oos.writeObject(p);
            }
            oos.close();
            System.out.println("\nRecords stored in Profile.bin");
            ObjectInputStream ois = new ObjectInputStream(new FileInputStream("Profile.bin"));
            Profile top = null;
            while (true) {
                try {
                    Profile p = (Profile) ois.readObject();
                    if (top == null || p.getAvgMarks() > top.getAvgMarks()) {
                        top = p;
                    }
                } catch (EOFException e) {
                    break;
                }
            }
            ois.close();
            System.out.println("\nHighest Scorer Details");
            System.out.println("Name : " + top.getName());
            System.out.println("Age : " + top.getAge());
            System.out.println("Average Marks : " + top.getAvgMarks());
        } catch (Exception e) {
            System.out.println(e);
        }
        sc.close();
    }
}

```


Output:

```
bappa@fedora:~/MCA_1year_2sem/Java/ass3$ javac Q17.java
bappa@fedora:~/MCA_1year_2sem/Java/ass3$ java Q17.java
Enter number of persons: 2

Person 1
Name: Bappa Das
Age: 22
Average Marks: 67

Person 2
Name: Mainak Mondal
Age: 23
Average Marks: 76

Records stored in Profile.bin

Highest Scorer Details
Name : Mainak Mondal
Age : 23
Average Marks : 76.0
bappa@fedora:~/MCA_1year_2sem/Java/ass3$
```

1. Write a prime generator program using only primes and using Python loops.

```
def is_prime(num):  
    if num < 2:  
        return False  
    for i in range(2, int(num ** 0.5) + 1):  
        if num % i == 0:  
            return False  
    return True
```

```
m = int(input("Enter the lower limit to generate primes up to: "))  
n = int(input("Enter the upper limit to generate primes up to: "))
```

```
primes = []  
for num in range(m, n + 1):  
    if is_prime(num):  
        primes.append(num)  
print(f"Primes up to {n}: {primes}")
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 1.py  
Enter the lower limit to generate primes up to: 20  
Enter the upper limit to generate primes up to: 50  
Primes up to 50: [23, 29, 31, 37, 41, 43, 47]  
bappa@fedora:~/MCA_1year_2sem/Python$
```

2. Write a discount coupon code using dictionary in Python with different rate coupons for each day of the week.

```
coupons = {  
    "Monday": ("MON10", 10),  
    "Tuesday": ("TUE15", 15),  
    "Wednesday": ("WED20", 20),  
    "Thursday": ("THU5", 5),  
    "Friday": ("FRI25", 25),  
    "Saturday": ("SAT30", 30),  
    "Sunday": ("SUN50", 50),  
}
```

```
day = input("Enter the day of the week (e.g. Monday): ").strip().capitalize()
```

```
if day in coupons:  
    code, discount = coupons[day]  
    print(f"Day : {day}")  
    print(f"Coupon : {code}")  
    print(f"Discount : {discount}%")
```

```
else:  
    print("Invalid day. Please enter a valid day name (e.g. Monday, Tuesday, ...).")
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 2.py  
Enter the day of the week (e.g. Monday): Friday  
Day : Friday  
Coupon : FRI25  
Discount : 25%  
bappa@fedora:~/MCA_1year_2sem/Python$
```

3. Print first 10 odd and even numbers using iterators and comprehension. You can use duck typing.

from itertools import compress, count, islice

```
# Generate first 20 numbers to ensure we get 10 of each
numbers = list(islice(count(0), 20))
```

```
# Selectors (masks)
evens_mask = [1, 0] * 10
odds_mask = [0, 1] * 10
```

```
evens = list(compress(numbers, evens_mask))
odds = list(compress(numbers, odds_mask))
```

```
print("Evens:", evens)
print("Odds:", odds)
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 3.py
Evens: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
Odds: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
bappa@fedora:~/MCA_1year_2sem/Python$
```

4. Write a regular expression to validate a phone number.

import re

```
def validate_phone(number):
    pattern = r"^\d{10}$" # Matches exactly 10 digits
    return bool(re.match(pattern, number))
```

Test

```
phone=input("Enter your phone number: ")
print(validate_phone(phone)) # True
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 4.py
Enter your phone number: 7797276158
True
bappa@fedora:~/MCA_1year_2sem/Python$ python3 4.py
Enter your phone number: 236589
False
bappa@fedora:~/MCA_1year_2sem/Python$
```

5. Write first seven Fibonacci numbers using generator next function/yield in Python. Trace and memorize the function.

```
def fibonacci_gen():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

```
fib = fibonacci_gen()
for _ in range(7):
    print(next(fib), end=" ")
print()
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 5.py
0 1 1 2 3 5 8
```

6. Write a simple program which loops over a list of user data and adds each user to a directory if valid. Raise exceptions for invalid username, age, email, etc.

```
import re

class UserDataError(Exception):
    """Base exception for all user data errors."""
    pass

class UsernameNotUniqueError(UserDataError):
    def __str__(self):
        return "Username is not unique (already exists in directory)."
```

class InvalidAgeError(UserDataError):

```
    def __str__(self):
        return "Age is not a positive integer."
```

class UnderageError(UserDataError):

```
    def __str__(self):
        return "User is under 16 years old."
```

class InvalidEmailError(UserDataError):

```
    def __str__(self):
        return "Email is not valid (must contain @ and a domain name)."
```

directory = {}

```
def validate_and_add(username, email, age):
    if username in directory:
        raise UsernameNotUniqueError()

    if not isinstance(age, int) or age <= 0:
        raise InvalidAgeError()

    if age < 16:
        raise UnderageError()

    email_pattern = r"^[^@]+@[^@]+\.[^@]+$"
    if not re.match(email_pattern, email):
        raise InvalidEmailError()

    # All checks passed
    directory[username] = {"email": email}
    print(f"{'{username}'} added to directory.")

print("Enter user data. Type 'done' as username to finish.\n")

users = []
while True:
    uname = input(" Username (or 'done'): ").strip()
    if uname.lower() == "done":
        break
    email = input(" Email : ").strip()
    age_input = input(" Age : ").strip()

    try:
        age = int(age_input)
    except ValueError:
```

```

    age = age_input # keep as non-int so InvalidAgeError triggers

    users.append((uname, email, age))

print("\n--- Processing Users ---")
for username, email, age in users:
    try:
        validate_and_add(username, email, age)
    except UsernameNotUniqueError as e:
        print(f"    [{username}] {e}")
    except InvalidAgeError as e:
        print(f"    [{username}] {e}")
    except UnderageError as e:
        print(f"    [{username}] {e}")
    except InvalidEmailError as e:
        print(f"    [{username}] {e}")

print(f"\nFinal Directory: {directory}")

```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 6.py
Enter user data. Type 'done' as username to finish.

Username (or 'done'): Bappa
Email   : bappa123@gmail.com
Age     : 22
Username (or 'done'): orthon
Email   : orthon23
Age     : 26
Username (or 'done'): modiji
Email   : modiji@modiji.in
Age     : 67
Username (or 'done'): done

--- Processing Users ---
'Bappa' added to directory.
[orthon] Email is not valid (must contain @ and a domain name).
'modiji' added to directory.

Final Directory: {'Bappa': {'email': 'bappa123@gmail.com'}, 'modiji': {'email': 'modiji@modiji.in'}}
bappa@fedora:~/MCA_1year_2sem/Python$

```

7. Write a function findfiles that recursively descends the directory tree and generates paths of all files.

```

import os

def findfiles(directory):
    """
    Recursively finds and generates paths of all files
    under the given directory tree.
    """
    for root, dirs, files in os.walk(directory):
        for filename in files:
            yield os.path.join(root, filename) # generator — yields one path at a time

path = input("Enter a directory path to scan (e.g. /home or .): ").strip()

if not os.path.isdir(path):
    print(f"'{path}' is not a valid directory.")
else:
    print(f"\nFiles found under '{path}':\n")
    count = 0
    for filepath in findfiles(path):
        print(f" {filepath}")

```

```
count += 1
```

```
print(f"\nTotal: {count} file(s) found.")
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 7.py
Enter a directory path to scan (e.g. /home or .): /home/bappa/MCA_1year_2sem/Python

Files found under '/home/bappa/MCA_1year_2sem/Python':

/home/bappa/MCA_1year_2sem/Python/1.py
/home/bappa/MCA_1year_2sem/Python/2.py
/home/bappa/MCA_1year_2sem/Python/3.py
/home/bappa/MCA_1year_2sem/Python/4.py
/home/bappa/MCA_1year_2sem/Python/5.py
/home/bappa/MCA_1year_2sem/Python/6.py
/home/bappa/MCA_1year_2sem/Python/7.py
/home/bappa/MCA_1year_2sem/Python/8.py
/home/bappa/MCA_1year_2sem/Python/9.py
/home/bappa/MCA_1year_2sem/Python/10.py

Total: 10 file(s) found.
bappa@fedora:~/MCA_1year_2sem/Python$
```

8. Create a list of all numbers up to N=50 which are multiples of five using anonymous function.

N = 50

```
numbers = range(1, N + 1)
```

```
multiples_of_five = list(filter(lambda x: x % 5 == 0, numbers))
```

```
print(f"Multiples of 5 up to N={N}:")
```

```
print(multiples_of_five)
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 8.py
Multiples of 5 up to N=50:
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
bappa@fedora:~/MCA_1year_2sem/Python$
```

9. Use map and zip to find the element-wise maximum among sequences of student list, subject list, and marks.

```
def get_list(name):
```

```
    raw = input(f"Enter {name} (space-separated numbers): ")
```

```
    return list(map(int, raw.split()))
```

```
student_list = get_list("Student List")
```

```
subject_list = get_list("Subject List")
```

```
marks_list = get_list("Marks List ")
```

```
result = list(map(lambda trio: max(trio), zip(student_list, subject_list, marks_list)))
```

```
print(f"\nStudent List : {student_list}")
```

```
print(f"Subject List : {subject_list}")
```

```
print(f"Marks List : {marks_list}")
```

```
print(f"Element-wise Max: {result}")
```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 9.py
Enter Student List (space-separated numbers): 1 2 3
Enter Subject List (space-separated numbers): 6 7 8
Enter Marks List (space-separated numbers): 67 76 86

Student List : [1, 2, 3]
Subject List : [6, 7, 8]
Marks List : [67, 76, 86]
Element-wise Max: [67, 76, 86]
bappa@fedora:~/MCA_1year_2sem/Python$

```

10. Filter out the odd squares using map, filter, and list.

```

raw = input("Enter numbers (space-separated): ")
numbers = list(map(int, raw.split()))

squares = list(map(lambda x: x ** 2, numbers))

odd_squares = list(filter(lambda x: x % 2 != 0, squares))

print(f"\nOriginal numbers : {numbers}")
print(f"Squared : {squares}")
print(f"Odd squares only : {odd_squares}")

```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 10.py
Enter numbers (space-separated): 6 7

Original numbers : [6, 7]
Squared : [36, 49]
Odd squares only : [49]
bappa@fedora:~/MCA_1year_2sem/Python$

```

11. List all Pythagorean triples whose short sides are numbers smaller than 10.

```

limit = int(input("Enter max value for short sides (e.g. 10): "))
triples_comp = [
    (a, b, c)
    for a in range(1, limit)
    for b in range(a, limit)
    for c in range(1, 200)
    if a ** 2 + b ** 2 == c ** 2
]
candidates = [
    (a, b, c)
    for a in range(1, limit)
    for b in range(a, limit)
    for c in range(1, 200)
]
triples_filter = list(
    filter(lambda t: t[0] ** 2 + t[1] ** 2 == t[2] ** 2, candidates)
)
print(f"\nPythagorean Triples with short sides < {limit}:")
print(f" Using Comprehension : {triples_comp}")
print(f" Using filter() : {triples_filter}")

```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 11.py
Enter max value for short sides (e.g. 10): 6

Pythagorean Triples with short sides < 6:
  Using Comprehension : [(3, 4, 5)]
  Using filter()      : [(3, 4, 5)]
bappa@fedora:~/MCA_1year_2sem/Python$

```

12. Enumerate the sequence of all lowercase ASCII letters starting from 1 using enumerate.

```
letters = [chr(c) for c in range(ord('a'), ord('z') + 1)]
```

```

print("Index Letter")
print("-" * 16)
for index, letter in enumerate(letters, start=1):
    print(f" {index:2d} {letter}")

```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 12.py
Index  Letter
-----
  1      a
  2      b
  3      c
  4      d
  5      e
  6      f
  7      g
  8      h
  9      i
 10      j
 11      k
 12      l
 13      m
 14      n
 15      o
 16      p
 17      q
 18      r
 19      s
 20      t
 21      u
 22      v
 23      w
 24      x
 25      y
 26      z
bappa@fedora:~/MCA_1year_2sem/Python$

```


13. Write a code which yields all terms of the geometric progression a , aq , aq^2 , aq^3 ... until the term exceeds 100000.

```
import time
```

```
def geo_gen(a, q):
```

```
    term = a
```

```
    loop_time_total = 0.0
```

```
    while True:
```

```
        if term > 100000:
```

```
            return loop_time_total
```

```
        loop_start = time.perf_counter()
```

```
        yield term
```

```
        loop_end = time.perf_counter()
```

```
        loop_time_total += (loop_end - loop_start)
```

```
        term = term * q
```

```
a = float(input("Enter the first term (a): "))
```

```
q = float(input("Enter the common ratio (q): "))
```

```
print(f"\nGeometric Progression: a={a}, q={q}")
```

```
print("Terms (stops when term > 100,000):\n")
```

```
total_start = time.perf_counter()
```

```
terms = []
```

```
gen = geo_gen(a, q)
```

```
loop_time = 0.0
```

```
try:
```

```
    while True:
```

```
        val = next(gen)
```

```
        terms.append(val)
```

```
        print(f" {val}")
```

```
except StopIteration as e:
```

```
    loop_time = e.value
```

```
total_end = time.perf_counter()
```

```
total_time = total_end - total_start
```

```
print(f"\nTotal terms generated : {len(terms)}")
```

```
print(f"Last term yielded : {terms[-1] if terms else 'None'}")
```

```
print(f"\nTotal time (seconds) : {total_time:.6f}s")
```

```
print(f"Time inside loop (sec) : {loop_time:.6f}s")
```

```
print(f"Time outside loop (sec) : {total_time - loop_time:.6f}s")
```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 13.py
Enter the first term (a): 4
Enter the common ratio (q): 5

Geometric Progression: a=4.0, q=5.0
Terms (stops when term > 100,000):

4.0
20.0
100.0
500.0
2500.0
12500.0
62500.0

Total terms generated      : 7
Last term yielded          : 62500.0

Total time (seconds)       : 0.000084s
Time inside loop (sec)     : 0.000053s
Time outside loop (sec)    : 0.000031s
bappa@fedora:~/MCA_1year_2sem/Python$

```

14. Search for palindrome and unique words in a text using class method and string method.

```

class TextAnalyzer:
    def __init__(self, text):
        self.text = text

    def palindrome_words(self):
        words = self.text.lower().split()
        palindromes = []

        for word in words:
            clean_word = ''.join(ch for ch in word if ch.isalnum())

            if clean_word == clean_word[::-1] and len(clean_word) > 1:
                if clean_word not in palindromes:
                    palindromes.append(clean_word)

        return palindromes

    def unique_words(self):
        words = self.text.lower().split()
        freq = {}

        for word in words:
            clean_word = ''.join(ch for ch in word if ch.isalnum())

            freq[clean_word] = freq.get(clean_word, 0) + 1

        unique = []

```

```

for word, count in freq.items():
    if count == 1:
        unique.append(word)

return unique

```

```
text = input("Enter a text: ")
```

```
obj = TextAnalyzer(text)
```

```
print("\nPalindrome Words:")
print(obj.palindrome_words())
```

```
print("\nUnique Words:")
print(obj.unique_words())
```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 14.py
Enter a text: madam sir madam

Palindrome Words:
['madam']

Unique Words:
['sir']
bappa@fedora:~/MCA_1year_2sem/Python$

```

15. Create a BankAccount class and a derived FeeSavingsAccount class with deposit, withdraw, balance inquiry, and fee deduction features.

```
class BankAccount:
```

```

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

```

```

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

```

```

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient Balance")

```

```

    def get_balance(self):
        print("Current Balance:", self.balance)

```

```

    def change_pin(self):
        old_pin = input("Enter old PIN: ")
        new_pin = input("Enter new PIN: ")
        print("PIN changed successfully")

```

```
class SavingsAccount(BankAccount):
```

```

    def __init__(self, name, balance, interest_rate, fee):
        super().__init__(name, balance)

```

```
self.interest_rate = interest_rate
self.fee = fee

def add_interest(self):
    interest = self.balance * self.interest_rate / 100
    self.balance += interest
    print("Interest Added:", interest)

def withdraw(self, amount):
    total = amount + self.fee

    if total <= self.balance:
        self.balance -= total
        print("Withdrawn:", amount)
        print("Fee Charged:", self.fee)
    else:
        print("Insufficient Balance")

name = input("Enter account holder name: ")
balance = float(input("Enter initial balance: "))
interest = float(input("Enter interest rate (%): "))
fee = float(input("Enter withdrawal fee: "))

acc = SavingsAccount(name, balance, interest, fee)

while True:
    print("\n1.Deposit")
    print("2.Withdraw")
    print("3.Add Interest")
    print("4.Check Balance")
    print("5.Change PIN")
    print("6.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        amt = float(input("Enter amount: "))
        acc.deposit(amt)

    elif choice == 2:
        amt = float(input("Enter amount: "))
        acc.withdraw(amt)

    elif choice == 3:
        acc.add_interest()

    elif choice == 4:
        acc.get_balance()

    elif choice == 5:
        acc.change_pin()

    elif choice == 6:
        break

    else:
        print("Invalid Choice")
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 15.py
```

```
Enter account holder name: bappa
```

```
Enter initial balance: 10000
```

```
Enter interest rate (%): 8
```

```
Enter withdrawal fee: 50
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 2
```

```
Enter amount: 500
```

```
Withdrawn: 500.0
```

```
Fee Charged: 50.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 4
```

```
Current Balance: 9450.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 1
```

```
Enter amount: 100
```

```
Deposited: 100.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 4
```

```
Current Balance: 9550.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 3
```

```
Interest Added: 764.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 4
```

```
Current Balance: 10314.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 5
```

```
Enter old PIN: 123
```

```
Enter new PIN: 365
```

```
PIN changed successfully
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 4
```

```
Current Balance: 10314.0
```

```
1.Deposit
```

```
2.Withdraw
```

```
3.Add Interest
```

```
4.Check Balance
```

```
5.Change PIN
```

```
6.Exit
```

```
Enter choice: 6
```

```
bappa@fedora:~/MCA_1year_2sem/Python$
```

16. Write an operator overloading function that returns the length of repetitive words in text and determines the most frequently occurring words.

```
from collections import Counter
class TextCheck:
    def __init__(self, text):
        self.text = text

    def __len__(self):
        words = self.text.lower().split()
        freq = Counter(words)
        repetitive = 0

        for count in freq.values():
            if count > 1:
                repetitive += 1

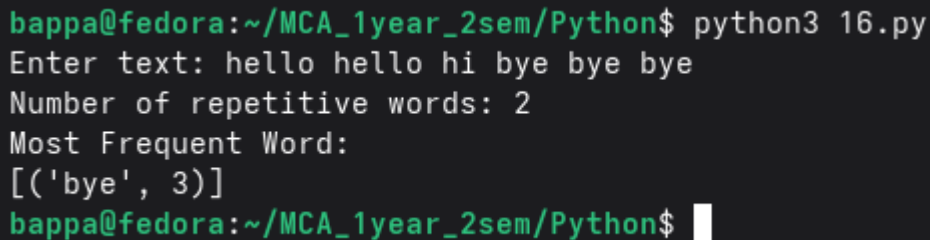
        return repetitive

    def most_common_word(self):
        words = self.text.lower().split()
        freq = Counter(words)

        return freq.most_common(1)
```

```
text = input("Enter text: ")
obj = TextCheck(text)

print("Number of repetitive words:", len(obj))
print("Most Frequent Word:")
print(obj.most_common_word())
```



```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 16.py
Enter text: hello hello hi bye bye bye
Number of repetitive words: 2
Most Frequent Word:
[('bye', 3)]
bappa@fedora:~/MCA_1year_2sem/Python$
```

17. Implement a priority queue that sorts items by priority and always returns the highest priority item first.

```
import heapq

pq = []
n = int(input("How many elements do you want to insert? "))

for i in range(n):
    item = input("Enter item: ")
    priority = int(input("Enter priority (smaller number = higher priority): "))

    heapq.heappush(pq, (priority, item))

print("\nItems removed based on priority:")

while pq:
    priority, item = heapq.heappop(pq)
    print(item, "with priority", priority)
```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 17.py
bappa@fedora:~/MCA_1year_2sem/Python$ python3 17.py
How many elements do you want to insert? 4
Enter item: 1
Enter priority (smaller number = higher priority): 1
Enter item: 2
Enter priority (smaller number = higher priority): 2
Enter item: 3
Enter priority (smaller number = higher priority): 4
Enter item: 4
Enter priority (smaller number = higher priority): 3

Items removed based on priority:
1 with priority 1
2 with priority 2
4 with priority 3
3 with priority 4
bappa@fedora:~/MCA_1year_2sem/Python$

```

18. Make a list of the largest or smallest N items in a collection.

```

import heapq

numbers = []

n = int(input("Enter number of elements: "))

for i in range(n):
    num = int(input("Enter number: "))
    numbers.append(num)

k = int(input("Enter value of N: "))

largest = heapq.nlargest(k, numbers)
smallest = heapq.nsmallest(k, numbers)

print("Largest", k, "items:", largest)
print("Smallest", k, "items:", smallest)

```

```

bappa@fedora:~/MCA_1year_2sem/Python$ python3 18.py
Enter number of elements: 4
Enter number: 1
Enter number: 2
Enter number: 3
Enter number: 3
Enter value of N: 2
Largest 2 items: [3, 3]
Smallest 2 items: [1, 2]
bappa@fedora:~/MCA_1year_2sem/Python$

```

19. Create dictionaries for stock prices, find minimum/maximum prices, compare dictionaries, and group prices by multiples of 500.

```
stock1 = {}
stock2 = {}

n1 = int(input("Enter number of stocks in first dictionary: "))

for i in range(n1):
    name = input("Enter stock name: ")
    price = int(input("Enter stock price: "))
    stock1[name] = price

print("\nFirst Dictionary:", stock1)

print("\nMinimum Price:", min(stock1.values()))
print("Maximum Price:", max(stock1.values()))

sorted_stock = dict(sorted(stock1.items(), key=lambda x: x[1]))

print("\nSorted by Price:")
print(sorted_stock)

n2 = int(input("\nEnter number of stocks in second dictionary: "))

for i in range(n2):
    name = input("Enter stock name: ")
    price = int(input("Enter stock price: "))
    stock2[name] = price

print("\nSecond Dictionary:", stock2)

print("\nStocks only in first dictionary:")

for key in stock1:
    if key not in stock2:
        print(key)

print("\nStocks with different prices:")

for key in stock1:
    if key in stock2 and stock1[key] != stock2[key]:
        print(key, ":", stock1[key], stock2[key])

print("\nIncrementing prices by 500:")

for key in stock1:
    stock1[key] += 500

print(stock1)

print("\nGrouping stocks by price multiple of 500:")

group = {}

for key, value in stock1.items():
    grp = (value // 500) * 500
```



```
if grp not in group:  
    group[grp] = []  
  
group[grp].append(key)  
  
print(group)
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 19.py  
Enter number of stocks in first dictionary: 3  
Enter stock name: ABC  
Enter stock price: 100  
Enter stock name: DEF  
Enter stock price: 200  
Enter stock name: GHI  
Enter stock price: 300  
  
First Dictionary: {'ABC': 100, 'DEF': 200, 'GHI': 300}  
  
Minimum Price: 100  
Maximum Price: 300  
  
Sorted by Price:  
{'ABC': 100, 'DEF': 200, 'GHI': 300}  
  
Enter number of stocks in second dictionary: 2  
Enter stock name: JKL  
Enter stock price: 250  
Enter stock name: XYZ  
Enter stock price: 150  
  
Second Dictionary: {'JKL': 250, 'XYZ': 150}  
  
Stocks only in first dictionary:  
ABC  
DEF  
GHI  
  
Stocks with different prices:  
  
Incrementing prices by 500:  
{'ABC': 600, 'DEF': 700, 'GHI': 800}  
  
Grouping stocks by price multiple of 500:  
{500: ['ABC', 'DEF', 'GHI']}  
bappa@fedora:~/MCA_1year_2sem/Python$
```

20. Write a function that flattens a nested dictionary structure like one obtained from Twitter and Facebook APIs.

```
def flatten_dict(d, parent_key="", sep="."):
    items = {}

    for k, v in d.items():
        new_key = parent_key + sep + k if parent_key else k

        if isinstance(v, dict):
            items.update(flatten_dict(v, new_key, sep=sep))

        elif isinstance(v, list):
            for i, item in enumerate(v):
                if isinstance(item, dict):
                    items.update(flatten_dict(item, f"{new_key} [{i}]", sep=sep))
                else:
                    items[f"{new_key} [{i}]"] = item
        else:
            items[new_key] = v

    return items

fullname = input("Enter full name: ")
age = int(input("Enter age: "))
phone_count = int(input("How many phone numbers? "))

phones = []
for i in range(phone_count):
    phones.append(input(f"Enter phone number #{i+1}: "))

first_line = input("Enter first-line address: ")
second_line = input("Enter second-line address: ")
zip_code = input("Enter ZIP code: ")

nested = {
    "fullname": fullname,
    "age": age,
    "phone-numbers": phones,
    "residence": {
        "address": {"first-line": first_line, "second-line": second_line},
        "zip": zip_code,
    },
}

print("\nNested Dictionary:")
print(nested)

flat = flatten_dict(nested)

print("\nFlattened Dictionary:")
for k, v in flat.items():
    print(f"{k} : {v}")
```

```
bappa@fedora:~/MCA_1year_2sem/Python$ python3 20.py
Enter full name: Bappa Das
Enter age: 22
How many phone numbers? 1
Enter phone number #1: 7797276158
Enter first-line address: Berhampore
Enter second-line address: Murshidabad
Enter ZIP code: 742165

Nested Dictionary:
{'fullname': 'Bappa Das', 'age': 22, 'phone-numbers': ['7797276158'], 'residence': {'address': {'first-line': 'Berhampore', 'second-line': 'Murshidabad'}, 'zip': '742165'}}

Flattened Dictionary:
fullname : Bappa Das
age : 22
phone-numbers[0] : 7797276158
residence.address.first-line : Berhampore
residence.address.second-line : Murshidabad
residence.zip : 742165
bappa@fedora:~/MCA_1year_2sem/Python$
```

